

Minimum-Time Optimal Control

2D Car & Bonus Case: Cart with Viscous Friction

Adrien BEAUCLAIR · Rayan CHENNAOUI · Nima KASHANI

CIRI Final Project — Detailed Technical Report

This report presents, in English, the full mathematical and numerical methodology behind our Optimal Control Project. It covers the OCP formulation, the Pontryagin Maximum Principle, the three-step numerical algorithm, and the bonus case (cart with viscous friction). Each tool and design choice is explained in detail.

1. Problem Formulation

1.1 What is an Optimal Control Problem?

An Optimal Control Problem (OCP) consists of finding a control function $u(t)$ that drives a dynamical system from an initial state to a target state while minimizing (or maximizing) some cost criterion. The general form is:

$$J(u) = h(X(T)) + \int_0^T f_0(t, X(t), u(t)) dt$$

where $h(X(T))$ is a terminal cost (paid once at the end) and f_0 is a running cost (accumulated over time). In our project, we want to minimize time, so:

$$J(u) = T = \int_0^T 1 dt \Rightarrow h = 0, f_0 = 1$$

1.2 The 2D Car (Dubins Car)

The Dubins car is a mathematical model of a vehicle moving in a plane at constant unit speed. At any instant its state is fully described by three quantities:

- **$x(t)$** : horizontal position (metres)
- **$y(t)$** : vertical position (metres)
- **$\theta(t)$** : heading angle (radians) — the direction the car is pointing

The control input $u(t)$ in $[-1, 1]$ represents the steering angle: $u = +1$ is maximum left turn, $u = -1$ is maximum right turn, $u = 0$ is straight ahead. The constant $k > 0$ scales the turning rate ($k = 1/R_{\min}$ where $R_{\min}$ is the minimum turning radius).

The system dynamics are:

$$\begin{aligned} dx/dt &= \cos(\theta(t)) \\ dy/dt &= \sin(\theta(t)) \\ d(\theta)/dt &= k * u(t) \end{aligned}$$

The full OCP is therefore: find u in $L_{\infty}([0, T], [-1, 1])$ minimising T , subject to the dynamics above, with initial state $(x, y, \theta)(0) = (0, 0, 0)$ and target state $(x, y, \theta)(T) = (x_f, y_f, \theta_f)$.

Why unit speed? Because minimising time T is then strictly equivalent to minimising the path length: every second the car travels exactly 1 metre. The difficulty comes from the heading constraint: we don't just want to reach (x_f, y_f) , we must arrive with the correct orientation θ_f .

1.3 State space and degrees of freedom

The state vector $X(t) = (x, y, \theta)^T$ lives in $\mathbb{R}^3$. The control $u(t)$ is a scalar in $[-1, 1]$. For a generic target in $\mathbb{R}^3$ (3 degrees of freedom), we need at least 3 free parameters — which is exactly what a Bang-Switch-Bang structure provides: two switching times t_1, t_2 , and the final time T .

2. Pontryagin Maximum Principle (PMP)

2.1 Overview

The Pontryagin Maximum Principle (PMP) is a necessary condition for optimality in control problems. It is the continuous-time analogue of Lagrange multipliers in finite-dimensional optimisation: it introduces co-state variables (also called adjoint variables or costates) that encode the sensitivity of the cost with respect to perturbations of the state trajectory.

The PMP tells us: instead of searching over all possible functions $u(t)$, we can characterise the optimal control by a set of differential equations and maximisation conditions. This is enormously powerful — it reduces an infinite-dimensional optimisation to a structured boundary value problem.

2.2 The Hamiltonian

We introduce the adjoint vector $p(t) = (p_x, p_y, p_{\theta})^T$ in $\mathbb{R}^3$, and the cost multiplier p^0 in $\{-1, 0\}$. The Hamiltonian is defined as:

$$H(t, X, u, p, p^0) = p^T * f(X, u) + p^0 * f_0(t, X, u)$$

Substituting the Dubins car dynamics and $f_0 = 1$:

$$H = p_x * \cos(\theta) + p_y * \sin(\theta) + p_{\theta} * k * u + p^0$$

The Hamiltonian has a clear physical meaning: it is the inner product of the costate p (sensitivity of cost to state) with the state velocity f , plus the running cost weighted by p^0 .

p^0 in $\{-1, 0\}$: when $p^0 = -1$, the problem is called normal — the cost is fully present in the Hamiltonian. When $p^0 = 0$, the problem is abnormal — the cost disappears, leaving only geometric constraints. Our problem is generically normal ($p^0 = -1$).

2.3 Adjoint equations

The PMP requires the costate to satisfy the adjoint ODE:

$$p_{\dot{}}(t) = -\text{grad}_X H(t, X^*, u^*, p^*, p^0)$$

Computing the partial derivatives of H with respect to (x, y, θ) :

$$p_{x\dot{}} = -dH/dx = 0 \Rightarrow p_x = \text{constant}$$

$$p_{y\dot{}} = -dH/dy = 0 \Rightarrow p_y = \text{constant}$$

$$p_{\theta\dot{}} = -dH/d(\theta) = p_x * \sin(\theta) - p_y * \cos(\theta)$$

The first two equations show that p_x and p_y are constants throughout the entire motion. The third shows that p_{θ} satisfies a nonlinear ODE driven by the state. Differentiating once more and using the dynamics of θ shows that p_{θ} is a sinusoid of $\theta(t)$, which can change sign at most twice on a bounded interval.

2.4 Maximality condition and bang-bang control

The PMP requires the optimal control u^* to maximise the Hamiltonian at every instant. Since H is linear in u :

$$u^*(t) = \text{argmax over } u \text{ in } [-1, 1] \text{ of } p_{\theta}(t) * k * u$$

Maximising a linear function over an interval always gives an extreme point. Therefore:

$$\begin{aligned}
u^*(t) &= +1 \text{ if } p_{\text{theta}}(t) > 0 \\
u^*(t) &= -1 \text{ if } p_{\text{theta}}(t) < 0 \\
u^*(t) &= 0 \text{ if } p_{\text{theta}}(t) = 0 \text{ (singular arc)}
\end{aligned}$$

This is called bang-bang control: the steering is always at maximum in one direction or the other, switching instantaneously when p_{theta} changes sign. Since p_{theta} is a sinusoid, it changes sign at most twice, giving a structure with at most two switching times — the BSB structure.

2.5 Proof that $(p_x, p_y) \neq 0$

The bang-bang result relies on $(p_x, p_y) \neq 0$. We prove this by contradiction. Assume $(p_x, p_y) = (0, 0)$. Then $p_{\text{theta_dot}} = 0$, so $p_{\text{theta}} = C$ (constant). The transversality condition (free final time) gives $H(T) = 0$, hence:

$$C * k * u^*(T) + p^0 = 0$$

Case $p^0 = 0$: then $C = 0$, so $p(t) = 0$ and $p^0 = 0$ simultaneously, violating the non-triviality condition of the PMP.

Case $p^0 = -1$: then $C * k * u^*(T) = 1$, so $C \neq 0$ and $u^*(T) = 1/Ck$. Since $u^*(T) \in \{-1, +1\}$, this forces $C = \pm 1/k$, meaning u^* is constant throughout — the car drives in a constant circle and can never reach a generic target. Contradiction.

Therefore $(p_x, p_y) \neq 0$, confirming the bang-bang structure.

2.6 Transversality condition for free T

Since the final time T is free (it is the quantity we minimise), the PMP imposes an additional condition:

$$H(T) = 0$$

This gives one extra equation that, combined with the three terminal state constraints (x_f, y_f, θ_f) , determines the three unknowns (t_1, t_2, T) of the BSB structure. The system is exactly determined — no redundancy, no under-determination.

3. Numerical Method: Three-Step Algorithm

The OCP is solved in three successive steps. Each step builds on the previous one, using its output as a warm start. This hierarchical strategy avoids local minima and ensures robust convergence.

3.1 Why discretise?

The OCP lives in continuous time: the control $u(t)$ is a function defined on $[0, T]$, an object of infinite dimension. A computer can only manipulate finite-dimensional objects. Discretisation is the process of replacing the continuous function by a finite vector of values evaluated at N time instants:

$$t_i = i * dt, \quad i = 0, \dots, N-1, \quad dt = T / (N-1)$$

The continuous ODE $\dot{x} = f(X, u)$ is approximated by the explicit Euler scheme:

$$X_{i+1} = X_i + f(X_i, u_i) * dt$$

For the 2D car, this gives the three recurrences:

$$\begin{aligned} x_{i+1} &= x_i + \cos(\theta_i) * dt \\ y_{i+1} &= y_i + \sin(\theta_i) * dt \\ \theta_{i+1} &= \theta_i + k * u_i * dt \end{aligned}$$

The optimisation vector then collects all state and control values:

$$X = (x_0, \dots, x_{N-1}, y_0, \dots, y_{N-1}, \theta_0, \dots, \theta_{N-1}, u_0, \dots, u_{N-2})$$

This is a vector in $\mathbb{R}^{(3N + N-1)} = \mathbb{R}^{(4N-1)}$. For $N=100$, that is 399 variables. The infinite-dimensional OCP has been reduced to a finite-dimensional nonlinear programme (NLP) that a numerical solver can handle.

Euler scheme accuracy: the Euler method is first-order — the error is $O(dt)$. With $N=100$ points over ~3 seconds, $dt = 0.03$ s, which is sufficient for our problem. Step 1 uses only $N=15$ (fast, coarse), Steps 2 and 3 use $N=100$ (precise).

3.2 Step 1 — Smooth reparameterisation of the bang-bang control

The PMP tells us the optimal control has at most two switching times $t_1 \leq t_2$ and a BSB structure. Instead of optimising over all 99 control values, we parameterise the entire control with only three scalars: $(\alpha_1, \alpha_2, s_2)$.

The switching times are encoded via sigmoids to automatically enforce the ordering $0 < t_1 < t_2 < T$ without any extra inequality constraints:

$$\begin{aligned} t_1 &= T * \text{sigma}(\alpha_1) \\ t_2 &= t_1 + (T - t_1) * \text{sigma}(\alpha_2) \\ \text{sigma}(x) &= 1 / (1 + \exp(-x)) \text{ in } (0, 1) \text{ for all } x \text{ in } \mathbb{R} \end{aligned}$$

For any (α_1, α_2) in $\mathbb{R}^2$, this automatically gives $0 < t_1 < t_2 < T$. No box constraints are needed on α_1, α_2 — the problem is unconstrained in those variables.

The smooth control profile uses a sigmoid-based blend of three arcs:

$$u_{\text{smooth}}(t) = \tanh(k_{\text{sig}} * (w_1(t) * (+1) + w_2(t) * s_2 + w_3(t) * (-1)))$$

where w_1, w_2, w_3 are sigmoid-based weighting functions that create smooth transitions between the three arcs. The $\tanh$ saturates the result toward ± 1 .

The objective at Step 1 is a saturation penalty:

$$J1 = \sum_{i=0}^{N-2} (1 - u_i^2) * dt$$

This quantity is always ≥ 0 , and equals 0 if and only if $u_i = \pm 1$ for all i . Minimising $J1$ therefore pushes the control toward the bang-bang extremes. The equality constraints enforce the terminal conditions $x(T) = x_f$, $y(T) = y_f$, $\theta(T) = \theta_f$.

Step 1 uses $N=15$ grid points intentionally. The goal is not accuracy — it is speed and robustness. A coarse grid is solved quickly and provides the correct BSB shape to initialise Step 2. The solver (SLSQP) may report success=False at Step 1: this is acceptable. What matters is that the switching structure is captured correctly.

3.3 Step 2 — Direct optimisation on the full trajectory (fixed T)

Using the Step 1 solution as a warm start, we now optimise over the full discretised trajectory with $N=100$ grid points. The optimisation vector is the 399-dimensional vector described in Section 3.1.

The final time T is fixed at a value slightly above the expected optimum (e.g. $T = 3.1$ s vs $T^* \sim 3.0$ s for the reference case). This ensures the problem is feasible: there exists at least one admissible trajectory. Searching for the optimal T is deferred to Step 3.

The proxy cost used at Step 2 is:

$$J2 = - \sum_{i=0}^{N-2} u_i^2 * dt$$

Minimising $J2$ maximises the sum of squared control values, penalising intermediate values near zero and rewarding saturation at ± 1 . This is consistent with the bang-bang structure predicted by the PMP.

The constraints are: (1) Euler dynamics at every step ($3*(N-1)$ equality constraints), (2) initial and terminal boundary conditions (6 equalities), (3) control bounds $-1 \leq u_i \leq 1$ ($N-1$ box constraints).

The warm start is built by interpolating the coarse Step 1 solution ($N=15$ points) onto the fine grid ($N=100$ points). This gives a starting point already close to the bang-bang structure, so the solver converges quickly without getting trapped in spurious local minima.

3.4 Step 3 — Free final time and tolerance continuation

The final step lifts the constraint on T by adding it as the last component of the optimisation vector:

$$X = (x_0, \dots, y_0, \dots, \theta_0, \dots, u_0, \dots, T) \text{ in } \mathbb{R}^{(4N)}$$

The true cost is now the final time itself, with a small regularisation term:

$$J3 = T - \epsilon * \sum_{i=0}^{N-2} u_i^2 * dt, \epsilon = 0.05$$

The regularisation term $-\epsilon * \sum u_i^2$ plays a technical role: without it, the solver might try to reduce T by setting $u = 0$ everywhere (the car goes straight, short path) and violate the heading constraint. The small penalty keeps the control active throughout.

A tolerance continuation scheme is used for robustness. Rather than directly solving at high precision (which the solver might fail to do from a distant starting point), we solve a sequence of increasingly tight problems:

$$\text{tol: } 1e-3 \rightarrow 1e-4 \rightarrow 1e-5 \rightarrow 1e-6$$

Each step reuses the previous solution as a warm start. This progressive approach guides the solver smoothly to the optimal solution without getting stuck in local minima.

3.5 Final refinement with fsolve

Once the SLSQP solution provides a good estimate of $(t1^*, t2^*, T^*)$, a last refinement is performed using `scipy.optimize.fsolve`. We exploit the exact BSB structure: the only unknowns are $(t1, t2, T)$, and the equations to satisfy are the three terminal conditions (x_f, y_f, θ_f) . This is a system of 3 equations in 3 unknowns — exactly determined.

`fsolve` solves this system to machine precision, giving a bang-bang control with exact switching times and a final time T^* accurate to at least 4 decimal places.

Why fsolve here and not SLSQP? fsolve is a root-finding algorithm (Newton-like) — it is faster and more precise than a constrained optimiser when the structure of the solution is fully known. SLSQP finds the shape; fsolve pins the exact switching times.

4. Python Tools and Their Roles

The following table summarises every Python function used in the notebook and explains what it does and why we chose it.

Tool / Function	Role	Why this choice
numpy	Array operations, trig functions, linspace, zeros	Standard scientific computing in Python; vectorised operations
scipy.optimize.minimize (method='SLSQP')	Solves the NLP at Steps 1, 2, 3	SLSQP handles both equality and inequality constraints; robust
scipy.optimize.fsolve	Exact BSB refinement at Step 3	Newton-like root finder; faster and more precise than SLSQP
scipy.integrate.solve_ivp (method='RK45')	High-accuracy ODE integration for Case 3 residual	RK45 is an adaptive-step Runge-Kutta method; much more accurate
matplotlib.pyplot	All plots (trajectory, control, heading)	Standard Python plotting library; supports step plots for bang-bang
sigma(x)	Sigmoid function $1/(1+\exp(-x))$	Maps $\mathbb{R} \rightarrow (0,1)$ smoothly; used to encode switching times with
smooth_control_vec	Constructs the sigmoid-based smooth BSB control	Provides a differentiable approximation of the bang-bang sign
rollout	Integrates the Dubins car dynamics (Euler)	Forward simulation of the state given a control sequence.
objective1/2/3	Cost functions J1, J2, J3	Each step uses a different proxy cost adapted to its purpose (
eq_constraints	Terminal boundary condition residuals	Passed to SLSQP as equality constraints; must return 0 at the
integrate_BSB	Exact bang-bang forward integration	Used in fsolve residual to evaluate the final state given exact
np.interp	Linear interpolation (coarse $\rightarrow$ fine grid)	Warm-starts Step 2 from Step 1 output; simple and fast for 1D

5. Results and Configurations

5.1 Reference case: $k=1$, target $(2, 2, \pi/2)$

For the reference configuration, the PMP predicts a B+B-B+ structure (left, right, left arcs). The numerically computed optimal time is:

$$T^* \sim 3.00 \text{ s}, \quad t_1^* \sim 1.19 \text{ s}, \quad t_2^* \sim 1.94 \text{ s}$$

The three-step algorithm recovers this solution to 4 decimal places. The control profile shows two clean switching instants with u in $\{-1, +1\}$ throughout, consistent with the bang-bang prediction.

5.2 Additional configurations studied

Three additional cases were studied to explore the sensitivity of the optimal solution to the problem parameters:

Configuration	k	Target (x_f, y_f, θ_f)	T^*	Structure	Key feature
Original	1.0	$(2, 2, \pi/2)$	$\sim 3.00 \text{ s}$	L+R-L+	Reference
Case 1	0.5	$(4, 4, \pi/2)$	$\sim 6.03 \text{ s}$	L+R-L+	Larger turning radius ($R=2\text{m}$); longer path needed
Case 2	1.0	$(2, -2, -\pi/2)$	$\sim 3.02 \text{ s}$	R-L+R-	Mirror geometry; symmetric RLR structure
Case 3	2.0	$(1.5, 1.5, \pi/2)$	$\sim 2.36 \text{ s}$	L+R-L+	Agile car ($R=0.5\text{m}$); tighter turns, shorter time

The most important observation is the effect of k on T^* : halving k (Case 1) doubles T^* , since the minimum turning radius doubles and longer arcs are needed. Conversely, doubling k (Case 3) allows tighter arcs and reduces T^* .

6. Bonus Case: Cart with Viscous Friction

6.1 Problem formulation

The bonus case studies a unit-mass cart on a frictionless rail, subject to viscous friction proportional to velocity. The state is $X(t) = (x(t), v(t))^T$ in $\mathbb{R}^2$, the control is $u(t)$ in $[-1, 1]$, and the friction coefficient is $\beta \geq 0$.

System dynamics:

$$\begin{aligned} dx/dt &= v(t) \\ dv/dt &= u(t) - \beta * v(t) \end{aligned}$$

OCP: minimise T subject to the dynamics, with $X(0) = (0, 0)^T$ and $X(T) = (d, 0)^T$ (start at rest, arrive at rest at position d).

Applications: train stopping at a station, spacecraft rendezvous, autonomous vehicle parking.

6.2 Hamiltonian and adjoint equations

The Hamiltonian is:

$$H(t, X, u, p, p^0) = p_x * v + p_v * (u - \beta * v) + p^0$$

The adjoint equations are:

$$\begin{aligned} p_{x_dot} &= -dH/dx = 0 \Rightarrow p_x = \text{constant} \\ p_{v_dot} &= -dH/dv = -p_x + \beta * p_v \end{aligned}$$

Unlike the frictionless cart (where p_v is linear), the second equation is now a first-order linear ODE with exponential solutions. Friction introduces a coupling between p_v and itself: the costate decays or grows exponentially depending on the sign of β .

6.3 Optimal control and switching structure

The maximality condition gives:

$$u^*(t) = \operatorname{argmax}_{u \in [-1,1]} p_v(t) * u = \operatorname{sign}(p_v(t))$$

The structure remains bang-bang. However, because p_v satisfies an exponential ODE (not linear as in the frictionless case), it can cross zero at most once on a bounded interval — giving a single switching time t^* .

Special case $\beta = 0$ (frictionless cart): p_v is linear in t and the analytical solution gives:

$$\begin{aligned} T^* &= 2 * \sqrt{d / u_{\max}}, \quad t^* = T^*/2 \text{ (symmetric switch)} \\ v_{\max} &= \sqrt{u_{\max} * d} \text{ (peak velocity at the switch)} \end{aligned}$$

The velocity profile is a symmetric triangle: maximum thrust followed by maximum braking, with the switch at exactly the midpoint in time.

6.4 Effect of friction ($\beta > 0$)

When $\beta > 0$, friction acts as a continuous dissipation of kinetic energy. To cover the same distance d in minimum time:

- The car must push harder against friction, so it uses the maximum thrust $u = +1$ for a longer fraction of the total time.
- The switch from push to brake occurs later than $T^*/2$ (asymmetric).
- The overall optimal time T^* increases with β : more friction $\Rightarrow$ more time.

From the numerical results: T^* grows from 2.000 s ($\beta=0$) to 2.007 s ($\beta=0.2$), 2.042 s ($\beta=0.5$), 2.170 s ($\beta=1.0$), and 2.689 s ($\beta=2.0$). This confirms the physical intuition: high friction significantly slows the cart and requires a longer trajectory.

6.5 Three-step algorithm adaptation

The same three-step algorithm is applied, with two modifications:

- **Step 1 — Two free variables:** Unlike the Dubins car (3 free parameters for 3 constraints), the cart has 1 switch and 2 state constraints. We therefore optimise over $(\alpha_{\text{sw}}, \alpha_T)$ in $\mathbb{R}^2$, encoding both the switching time and the total time via sigmoids.
- **fsolve refinement:** The exact BSB system is 2 equations in 2 unknowns (t^*, T) — the two terminal conditions $x(T) = d$ and $v(T) = 0$. fsolve finds the exact solution.

The costate p_v can be recovered analytically from the switching condition $p_v(t^) = 0$ and the transversality condition $H(T) = 0$. One then verifies that $H(t) = 0$ throughout the trajectory (up to Euler discretisation error), confirming the optimality of the computed solution.*

7. Neural ODE: System Identification

7.1 The inverse problem

System identification is the inverse of simulation. Instead of knowing the dynamics and computing trajectories, we observe trajectories and try to recover the dynamics that produced them. Concretely, given N_{data} pairs $(x0_i, x1_i)$ where $x0_i$ is the state at time $t0$ and $x1_i$ is the state 1 second later, we want to find a function $f_w: \mathbb{R}^3 \rightarrow \mathbb{R}^3$ such that integrating $\dot{x} = f_w(x)$ from $x0_i$ gives a trajectory close to $x1_i$.

In our application, the true dynamics to identify is the Dubins car with fixed control $u0 = +1$:

$$f(x, y, \theta) = (\cos(\theta), \sin(\theta), k \cdot u0)^T = (\cos(\theta), \sin(\theta), 1)^T$$

The loss function to minimise is:

$$L(w) = \sum_{i=1}^{N_{\text{data}}} (1/2) * || x_w(t1_i; x0_i) - x1_i ||^2$$

7.2 Neural network architecture

The function f_w is modelled by a fully connected neural network (MLP) with:

- Input layer: dimension 3 (state x, y, θ)
- Hidden layer 1: 32 neurons, tanh activation
- Hidden layer 2: 32 neurons, tanh activation
- Output layer: dimension 3, linear activation (no saturation)

Total parameters: $(3 \times 32 + 32) + (32 \times 32 + 32) + (32 \times 3 + 3) = 128 + 1056 + 99 = 1283$.

The output layer is initialised with $W_{\text{out}} = 0$, $b_{\text{out}} = (0, 0, 1)^T$. This means at the start of training, $f_w(x) = (0, 0, 1)^T$ regardless of x , which is the mean of the true dynamics averaged over all headings. This intelligent initialisation dramatically accelerates convergence.

7.3 The adjoint method for gradient computation

Computing the gradient dL/dw requires differentiating through an ODE integration. The adjoint method does this efficiently in three steps:

Step 1 — Forward pass: Integrate $\dot{x} = f_w(x)$ forward in time using Euler with $N_{\text{int}} = 50$ steps. Store the trajectory $\{x_k\}$ and the Jacobians $df_w/dx(x_k)$ and $df_w/dw(x_k)$ at each step.

Step 2 — Backward pass: Define the adjoint variable $p(t)$ in $\mathbb{R}^3$ with terminal condition $p(t1) = x_w(t1) - x1$ (the prediction error). Integrate backward in time:

$$\begin{aligned} p_{\text{dot}}(t) &= -(df_w/dx(x(t)))^T * p(t) \\ p_{\{k-1\}} &= p_k - (df_w/dw(x_k))^T * p_k * dt \end{aligned}$$

The costate $p(t)$ measures at each past instant how a small perturbation of the state at that instant propagates to the final error.

Step 3 — Gradient by quadrature:

$$\text{grad}_w L = \sum_{k=0}^{N_{\text{int}}-2} (df_w/dw(x_k))^T * p_k * dt$$

This is computed using the Jacobians stored during the forward pass and the adjoint values computed during the backward pass.

The adjoint method is mathematically equivalent to backpropagation in neural networks, but applied to an ODE integrator instead of a static computational graph. It avoids storing the entire history of intermediate gradients, making it memory-efficient.

7.4 Training with Stochastic Gradient Descent

The weights are updated by SGD with mini-batches of size 5 from the $N_{\text{data}} = 20$ training trajectories:

$$w_{\{k+1\}} = w_k - h * \sum_{\{j \text{ in } B_k\}} \text{grad}_w L_j(w_k), \quad h = 3e-3$$

Mini-batching introduces noise that helps escape shallow local minima. The learning rate $h = 3e-3$ was chosen empirically: too large causes divergence, too small requires too many iterations.

Analytical Jacobians replace automatic differentiation (autograd) in the inner loop, achieving a 10-50x speed-up and allowing 1000 training iterations in reasonable time.

8. Summary

This project demonstrates a complete pipeline for solving minimum-time optimal control problems, combining analytical theory (PMP) with robust numerical methods.

Component	Key idea	Main result
OCP formulation	Minimise T subject to Dubins car dynamics	3-state system, 1 scalar control in $[-1,1]$
PMP — Hamiltonian	$H = p_x \cos(\theta) + p_y \sin(\theta) + p_\theta * k * u$	Linear in $u \Rightarrow$ bang-bang optimal control
Adjoint equations	p_x, p_y constant; p_θ sinusoidal	At most 2 sign changes $\Rightarrow$ at most 2 switches
Bang-bang proof	Contradiction on $(p_x, p_y) = 0$	Proves non-singularity; confirms BSB structure
Step 1	Sigmoid reparameterisation (α_1, α_2, s_2)	Warm start encoding correct BSB shape
Step 2	Full trajectory NLP at fixed $T, N=100$	Near-perfect bang-bang, all constraints satisfied
Step 3	Free T + tolerance continuation + fsolve	T^* accurate to 4+ decimal places
Bonus: cart	Double integrator with friction (β)	T^* grows with β ; single switch at $t^* > T^*/2$
Neural ODE	MLP learns Dubins dynamics from 20 trajectories	Adjoint method + SGD identifies $f_w(x)$